Snyk Code (SAST)

Target: orangehrm/orangehrm (main)
Project: orangehrm
Total Issues: 187 (Critical: 0 | High: 1 | Medium: 11 | Low: 175)

HIGH (1)

HIGH

Hardcoded Non-Cryptographic Secret

Location `src/client/src/orangehrmCoreOAuthPlugin/pages/registerOAuthClient/SaveOAuthClient.vue:128`

Rule javascript/HardcodedNonCryptoSecret

Avoid hardcoding values that are meant to be secret. Found a hardcoded string used in here.

Details

Details

When constants are hardcoded into applications, this information could easily be reverse-engineered and become known to attackers. For example, if a breached authentication token is hardcoded in multiple places in the application, it may lead to components of the application remaining vulnerable if not all instances are changed.
Another negative effect of hard-coding constants is potential unpredictability in the application's performance if the development team fails to update every single instance of the hardcoded constant throughout the code. For these reasons, hard-coding security-relevant constants is considered bad coding practice and should be remedied if present and avoided in future.

Best practices for prevention
- Never hard code security-related constants; use symbolic names or configuration lookup files.
- As hard coding is often done by coders working alone on a small scale, examine all legacy code components and test carefully when scaling.
- Adopt a "future-proof code" mindset: While use of constants may save a little time now and make development simpler in the short term, it could cost time and money adapting to scale or other unforeseen circumstances (such as new hardware) in the future.

MEDIUM (11)

MEDIUM

Use of Hardcoded Passwords

Location `installer/Util/AppSetupUtility.php:322`

Rule php/HardcodedPassword

Do not hardcode passwords in code. Found a hardcoded password used in variable.

Details

Details

Developers may use hardcoded passwords during development to streamline setup or simplify authentication while testing. Although these passwords are intended to be removed before deployment, they are sometimes inadvertently left in the code. This introduces serious security risks, especially if the password grants elevated privileges or is reused across multiple systems.

An attacker who discovers a hardcoded password can potentially gain unauthorized access, escalate privileges, exfiltrate sensitive data, or disrupt service availability. If the password is reused across different environments or applications, the compromise can spread quickly and broadly.

Best practices for prevention

* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

MEDIUM

Information Exposure - Server Error Message

Location `devTools/load/general/load-employees.php:32`

Rule php/ServerLeak

An MySQLi error object flows to the echo statement and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.

Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention

* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

MEDIUM

Information Exposure - Server Error Message

Location `devTools/load/general/load-employees.php:38`

Rule php/ServerLeak

An MySQLi error object flows to the echo statement and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.
Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention
* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

MEDIUM

Information Exposure - Server Error Message

Location `devTools/load/general/load-employees.php:39`

Rule php/ServerLeak

An MySQLi error object flows to the echo statement and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.
Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention
* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

MEDIUM

Information Exposure - Server Error Message

Location `devTools/load/general/load-employees.php:40`

Rule php/ServerLeak

An MySQLi error object flows to the echo statement and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.
Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention
* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

MEDIUM

Information Exposure - Server Error Message

Location `devTools/load/general/load-employees.php:314`

Rule php/ServerLeak

An MySQLi error object flows to the echo statement and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.
Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention
* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

MEDIUM

Information Exposure - Server Error Message

Location `devTools/load/general/load-employees.php:320`

Rule php/ServerLeak

An MySQLi error object flows to the echo statement and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.
Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention
* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

MEDIUM

Information Exposure - Server Error Message

Location `devTools/load/general/load-employees.php:333`

Rule php/ServerLeak

An MySQLi error object flows to the echo statement and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.
Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention
* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

MEDIUM

Information Exposure - Server Error Message

Location `devTools/load/general/load-employees.php:336`

Rule php/ServerLeak

An MySQLi error object flows to the echo statement and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.
Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention
* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

MEDIUM

Information Exposure - Server Error Message

Location `devTools/load/recruitment/load-candidates.php:96`

Rule php/ServerLeak

An MySQLi error object flows to the echo statement and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.
Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention
* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

MEDIUM

Information Exposure - Server Error Message

Location `installer/index.php:70`

Rule php/ServerLeak

An exception object flows to die/exit and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.
Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention

* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

LOW (175)

LOW

Use of Hardcoded Credentials

Location `src/test/functional/cypress/e2e/integration/feature/admin/workspaceNotification.cy.js:121`

Rule javascript/NoHardcodedCredentials/test

Do not hardcode credentials in code. Found hardcoded credential used in Slack Webhook URL.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention

* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Information Exposure - Server Error Message

Location `src/test/phpunit/Util/TestDataService.php:104`

Rule php/ServerLeak/test

An exception object flows to the echo statement and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.

Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention

* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

LOW

Information Exposure - Server Error Message

Location `src/test/phpunit/Util/TestDataService.php:300`

Rule php/ServerLeak/test

An exception object flows to the echo statement and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.
Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention
* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

LOW

Information Exposure - Server Error Message

Location `src/test/phpunit/Util/bootstrap.php:48`

Rule php/ServerLeak/test

An exception object flows to the echo statement and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.
Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention
* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

LOW

Information Exposure - Server Error Message

Location `src/test/phpunit/Util/bootstrap.php:54`

Rule php/ServerLeak/test

An exception object flows to die/exit and is leaked to the attacker. This may disclose important information about the application to an attacker.

Details

Details

Sensitive data includes: personally identifiable information (PII) of employees, customers, users, or other third parties; financial information; code; intellectual property; or protected network information such as passwords. Exposure of sensitive information to an unauthorized actor occurs when any party who does not require this information for business purposes can remove this information from the secure network.
Consequences of exposure can include holding it for ransom payment, identity theft, or access to other internal network resources-along with financial loss due to regulatory fines, court settlements, or cost to restore compromised systems. As most attackers aim to extract sensitive information, this CWE only describes exploits that take advantage of vulnerabilities related to how data is handled: the management, storage, transfer, or removal of sensitive information.

Best practices for prevention
* Encrypt data at all times, whether in transit or at rest.
* When transporting data, always use a secure protocol such as TLS.
* Before collecting PII or financial information, consider whether there is a true business need for this information; similarly, before storing the data, ensure that there is an ongoing business need.
* Classify all incoming data according to privacy and security guidelines to improve awareness of sensitive data.
* Ensure that caching is disabled for all transactions involving sensitive information.
* Store passwords with secure, salted hashing functions.
* Apply a network-wide least-privilege policy so sensitive data is provided only to users with business need to know.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmAdminPlugin/test/Dao/UserDaoTest.php:75`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmAdminPlugin/test/Service/UserServiceTest.php:69`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmAdminPlugin/test/Service/UserServiceTest.php:94`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmAdminPlugin/test/Service/UserServiceTest.php:146`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmAdminPlugin/test/Service/UserServiceTest.php:169`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmAdminPlugin/test/Service/UserServiceTest.php:196`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmAdminPlugin/test/Service/UserServiceTest.php:224`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmAdminPlugin/test/Service/UserServiceTest.php:272`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmAdminPlugin/test/Service/UserServiceTest.php:316`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmAdminPlugin/test/Service/UserServiceTest.php:354`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmAuthenticationPlugin/test/Dao/LoginLogDaoTest.php:57`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmAuthenticationPlugin/test/Service/LoginLogServiceTest.php:135`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmPimPlugin/test/Api/EmploymentContractAPITest.php:380`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmPimPlugin/test/Api/EmploymentContractAPITest.php:694`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUserName.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmAuthenticationPlugin/test/Service/ResetPasswordServiceTest.php:181`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUsername.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Dto/LDAPUserTest.php:38`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUsername.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Dto/LDAPUserTest.php:55`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUsername.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmOpenidAuthenticationPlugin/test/Service/SocialMediaAuthenticationServiceTest.php:101`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUsername.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmOpenidAuthenticationPlugin/test/Service/SocialMediaAuthenticationServiceTest.php:107`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUsername.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmOpenidAuthenticationPlugin/test/Service/SocialMediaAuthenticationServiceTest.php:116`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUsername.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Credentials

Location `src/plugins/orangehrmOpenidAuthenticationPlugin/test/Service/SocialMediaAuthenticationServiceTest.php:143`

Rule php/HardcodedCredential/test

Do not hardcode credentials in code. Found a hardcoded credential used in setUsername.

Details

Details

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

Once attackers gain access, they may take advantage of privilege level to remove or alter data, take down a site or app, or hold any of the above for ransom. The risk across multiple similar projects is even greater. If code containing the credentials is reused across multiple projects, they will all be compromised.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Hardcoded Passwords

Location `src/plugins/orangehrmAuthenticationPlugin/test/Service/ResetPasswordServiceTest.php:182`

Rule php/HardcodedPassword/test

Do not hardcode passwords in code. Found a hardcoded password used in setPassword.

Details

Details

Developers may use hardcoded passwords during development to streamline setup or simplify authentication while testing. Although these passwords are intended to be removed before deployment, they are sometimes inadvertently left in the code. This introduces serious security risks, especially if the password grants elevated privileges or is reused across multiple systems.

An attacker who discovers a hardcoded password can potentially gain unauthorized access, escalate privileges, exfiltrate sensitive data, or disrupt service availability. If the password is reused across different environments or applications, the compromise can spread quickly and broadly.

Best practices for prevention
* Plan software architecture such that keys and passwords are always stored outside the code, wherever possible.
* Plan encryption into software architecture for all credential information and ensure proper handling of keys, credentials, and passwords.
* Prompt for a secure password on first login rather than hard-code a default password.
* If a hardcoded password or credential must be used, limit its use, for example, to system console users rather than via the network.
* Use strong hashes for inbound password authentication, ideally with randomly assigned salts to increase the difficulty level in case of brute-force attack.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:155`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:156`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:157`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:158`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:159`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:160`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:161`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:162`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:163`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:164`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:165`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:166`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:167`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:168`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:169`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:170`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:171`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:172`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:173`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:174`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:175`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:176`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:177`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:178`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:179`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:180`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:181`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:182`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:183`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:184`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:185`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:186`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:187`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:188`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:189`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:190`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:191`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:192`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:193`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:194`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:195`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:196`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:197`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:198`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:199`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:200`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:201`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:202`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:203`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:204`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:205`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:206`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:207`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:208`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:209`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:210`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:211`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:212`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:213`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:214`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:215`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:216`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:217`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:218`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:219`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:220`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:221`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:222`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:223`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:224`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:225`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:226`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:227`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:228`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:229`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:230`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:231`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:232`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:233`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:234`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:235`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:236`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:237`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:238`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:239`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:240`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:241`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:242`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:243`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:244`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:245`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:246`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:247`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:248`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:249`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:250`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:251`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:252`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:253`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:254`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:255`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:256`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:257`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:258`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:259`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:260`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:261`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:262`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:263`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `devTools/load/general/load-employees.php:264`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmAdminPlugin/Service/UserService.php:245`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmCorePlugin/Service/ReportGeneratorService.php:337`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmCorePlugin/Utility/KeyHandler.php:44`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/Service/LDAPSyncService.php:161`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/Service/LDAPSyncService.php:407`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLeavePlugin/Service/AbstractLeaveAllocationService.php:92`

Rule php/InsecureHash

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmAdminPlugin/test/Service/UserServiceTest.php:191`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmAdminPlugin/test/Service/UserServiceTest.php:310`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Api/LDAPTestConnectionAPITest.php:246`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceCreateSystemUsersTest.php:262`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceDeleteUsersTest.php:317`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceDeleteUsersTest.php:340`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceDeleteUsersTest.php:350`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceDeleteUsersTest.php:360`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceDeleteUsersTest.php:370`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceDeleteUsersTest.php:397`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceDeleteUsersTest.php:407`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceDeleteUsersTest.php:417`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:181`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:189`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:197`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:206`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:215`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:223`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:231`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:240`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:249`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:257`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:265`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:273`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:281`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:289`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:297`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:305`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:314`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:323`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:332`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention

* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.

LOW

Use of Password Hash With Insufficient Computational Effort

Location `src/plugins/orangehrmLDAPAuthenticationPlugin/test/Service/LDAPSyncServiceFieldValidationTest.php:340`

Rule php/InsecureHash/test

MD5 hash (used in md5) is insecure. Consider changing it to a secure hashing algorithm.

Details

Details

Sensitive information should never be stored in plain text, since this makes it very easy for unauthorized users, whether malicious insiders or outside attackers, to access. Hashing methods are used to make stored passwords and other sensitive data unreadable to users. For example, when a password is defined for the first time, it is hashed and then stored. The next time that user attempts to log on, the password they enter is hashed following the same procedure and compared with the stored value. In this way, the original password never needs to be stored in the system.

Hashing is a one-way scheme, meaning a hashed password cannot be reverse engineered. However, if an outdated or custom programmed hashing scheme is used, it becomes simple for an attacker with powerful modern computing power to gain access to the hashes used. This opens up access to all stored password information, leading to breached security. Therefore, it is essential for developers to understand modern, secure password hashing techniques.

The strength of this scheme depends on the hashing algorithm. Fast, general-purpose hashes such as MD5 or SHA-1, and hashes computed without a per-password salt, can be brute-forced or attacked with precomputed tables using modern hardware. Verifying a password against such a hash (for example storedhash == md5(password).hexdigest()) leaves stored credentials exposed if the hash database is ever leaked.

Best practices for prevention
* Use strong standard algorithms for hashing rather than simpler but outdated methods or DIY hashing schemes, which may have inherent weaknesses.
* Hash passwords with a slow, computationally expensive algorithm designed for passwords, such as bcrypt, scrypt, Argon2, or PBKDF2. Do not use fast general-purpose hashes (MD5, SHA-1, SHA-256) for password storage.
* Use modular design for all code dealing with hashing so it can be swapped out as security standards change over time.
* Always use a unique, random salt per password so identical passwords do not produce identical hashes (While this places more demands on resources, it is an essential step for tighter security.).
* Implement zero-trust architecture to ensure that access to password data is granted only for legitimate business purposes.
* Increase developer awareness of current standards in data security and cryptography.